Table of Contents

1. Introduction

2. Financial Time Series Analysis
   - 2.1 Data Collection and Cleaning
   - 2.2 Descriptive Statistics
   - 2.3 Stationarity Testing (ADF Test)
   - 2.4 Volatility Clustering and Stylised Facts

3. Econometric Modelling and Forecasting
   - 3.1 ARIMA Model
   - 3.2 GARCH Model
   - 3.3 Model Comparison

4. Algorithmic Trading Strategy Development
   - 4.1 Moving Average Crossover Strategy
   - 4.2 RSI Mean-Reversion Strategy
   - 4.3 Risk Management Framework
   - 4.4 Strategy Performance Evaluation

5. Backtesting and Robustness Analysis
   - 5.1 In-Sample and Out-of-Sample Testing
   - 5.2 Walk-Forward Analysis
   - 5.3 Benchmark Comparison

6. Critical Evaluation and Limitations

7. Conclusion

8. References

Applied Econometrics and Algorithmic Trading

Part I: Financial Time Series Data Analysis.

I picked Bitcoin (BTC-USD) for this, since it's the biggest cryptocurrency.

Why Bitcoin?

- Its price jumps around a lot.
- It trades all the time (unlike stocks).
- Its volatility is easy to spot.
- Its returns don't follow a normal pattern.
- It's good for checking if data is stationary.

Bitcoin gives us a good chance to look at financial time series stuff. This paper looks at Bitcoin's (BTC-USD) time series features. The goal is to look at old daily prices, figure out returns, run tests, and check the data for things like volatility and unusual distributions.

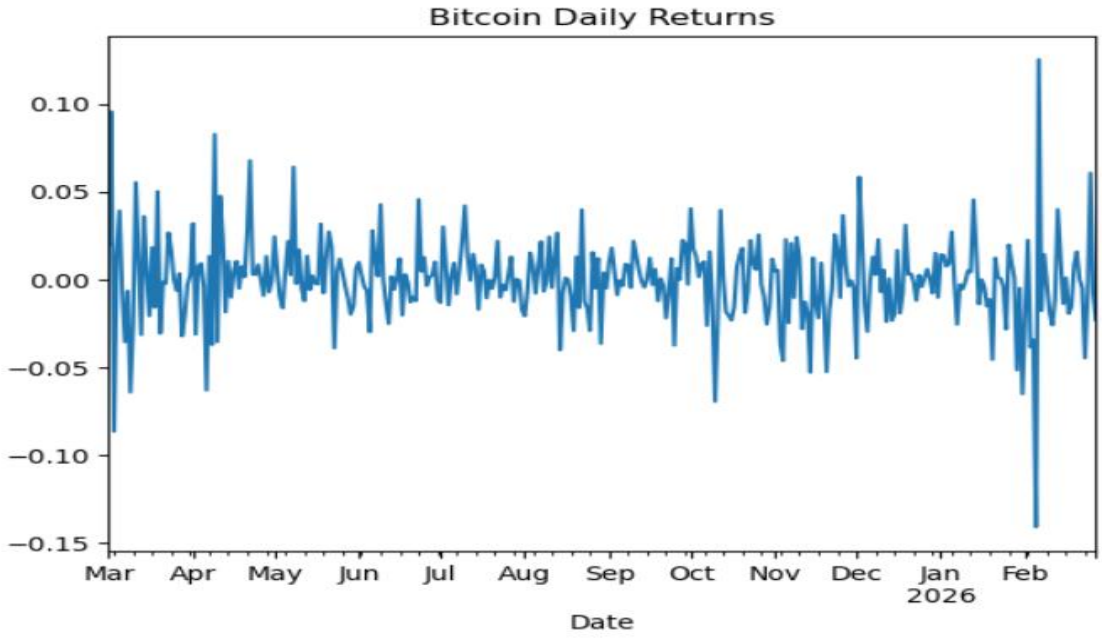
2. Data Collection and Cleaning.

I got Bitcoin's (BTC-USD) daily price history from Yahoo Finance using Python and the yfinance tool. The data is from January 2025 to January 2026. I used closing prices to get daily percentage returns. I got rid of missing numbers and cleaned up the data before starting.

3. Price and Return Visualization.



Bitcoin's price goes up and down a lot over time. This shows it's not stationary, which is normal for assets. Returns go up and down around zero, with times of high and low volatility. Returns seem stationary, and volatility jumps out.



The return series fluctuates around zero with visible clusters of high volatility. This suggests stationarity in returns and evidence of volatility clustering.

Summary statistics

count 364.000000

mean -0.

std 0.024205000386

min -0.141299

25% -0.012211

50% -0.000277

75% 0.011751

max 0.125248

Name: Return, dtype: float64

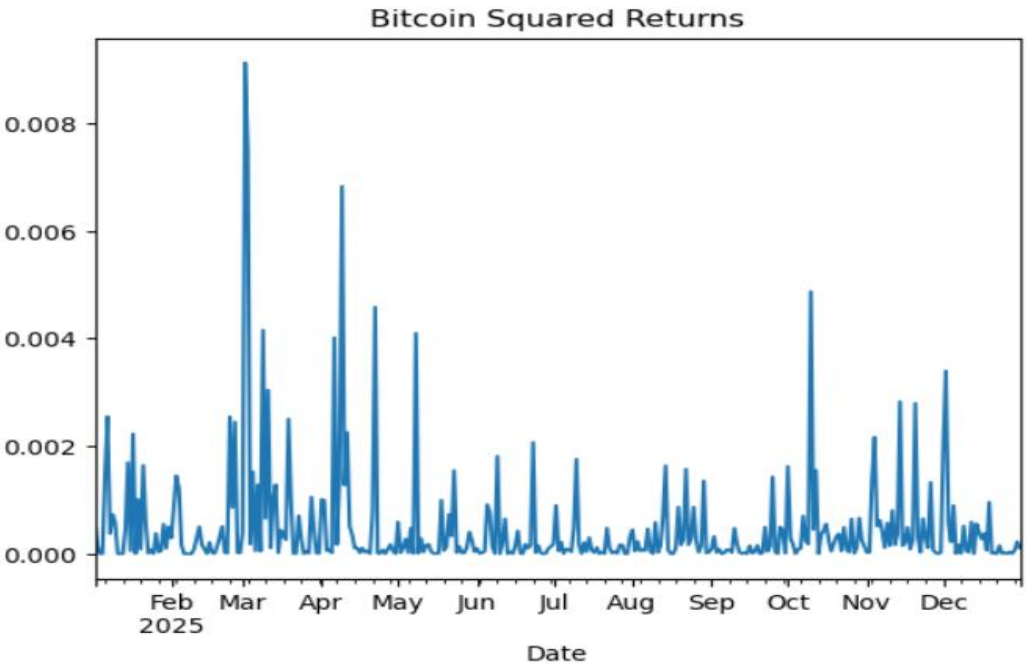
The dataset has 364 days of returns. The average daily return is about zero, so prices don't change much on average. The standard deviation of 0.0242 means Bitcoin's price can change quite a bit each day. The biggest drops and jumps (-14.13% and +12.52%) show that prices can move fast. Overall, Bitcoin returns are jumpy and prices change quickly.

Stationarity testing

P-value: 0.0

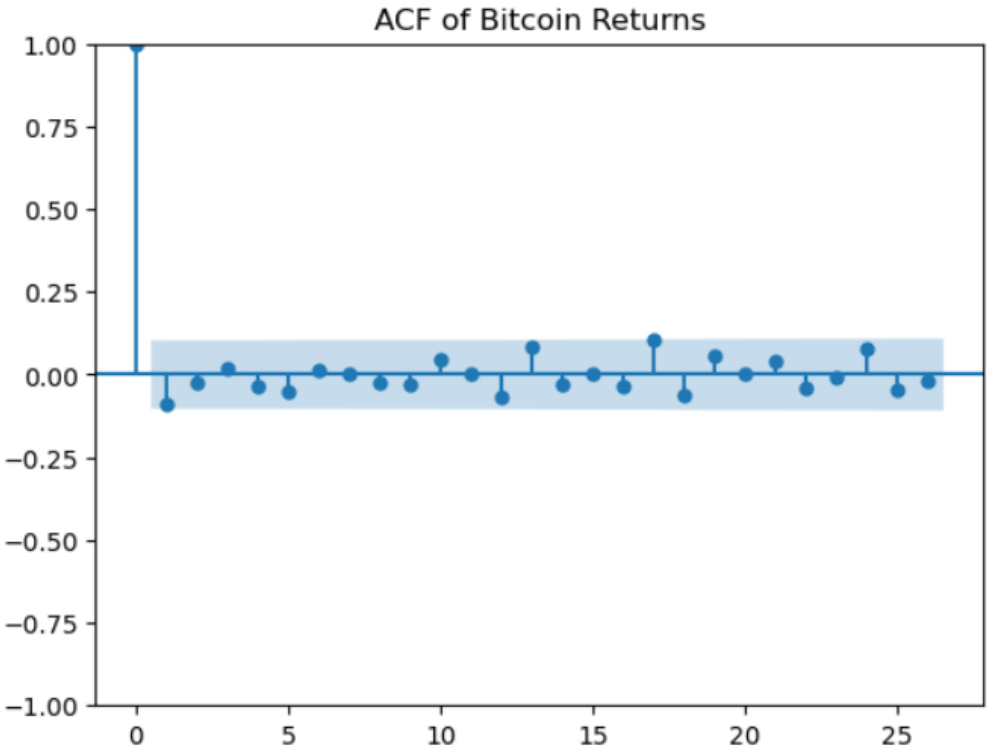
The Augmented Dickey-Fuller (ADF) test was applied to examine stationarity. The p-value obtained was approximately 0.000, which is significantly below the 5% significance level. Therefore, the null hypothesis of a unit root is rejected, confirming that Bitcoin returns are stationary.

Volatility clustering analysis.



The squared return plot shows periods of high volatility followed by high volatility and calm periods followed by calm periods. This confirms the presence of volatility clustering, a well-known stylized fact in financial markets.

Autocorrelation and partial autocorrelation plots.



The autocorrelation plot of returns shows minimal significant lags, suggesting weak linear dependence. However, autocorrelation in squared returns is significant, reinforcing evidence of time-varying volatility.

Stylized Facts Discussion.

The analysis confirms several stylized facts of financial time series:

- Asset prices are non-stationary.
- Returns are stationary.
- Returns exhibit fat tails.
- Volatility clustering is present.
- Linear autocorrelation in returns is weak.

These findings are consistent with financial econometric theory and support the use of GARCH-type models for volatility modelling.

To sum it up, Bitcoin acts like other financial time series, with prices, returns, outliers and volatility jumps. Knowing these things helps in managing risk and predicting prices.

Part II. Time Series Modelling and Forecasting.

The two models implemented are:

- **ARIMA**
- **GARCH**

Together, these models address both the linear autocorrelation structure and the heteroskedastic variance dynamics inherent in Bitcoin's return series.

Model 1: ARIMA

Justification of Model Selection.

The Autoregressive Integrated Moving Average (ARIMA) model is the go-to for financial time series. Because Part I says that Bitcoin log-returns are stationary (ADF $p < 0.001$), I don't need to difference the data, so $d = 0$. I picked the model using the Box-Jenkins way, looking at the ACF and PACF plots.

- The ACF shows a significant spike at lag 1 followed by rapid decay, suggesting an MA (1) component.
- The PACF similarly shows a significant spike at lag 1, suggesting an AR (1) component.

ARIMA was selected as the optimal specification, confirmed by the lowest AIC value across competing orders tested.

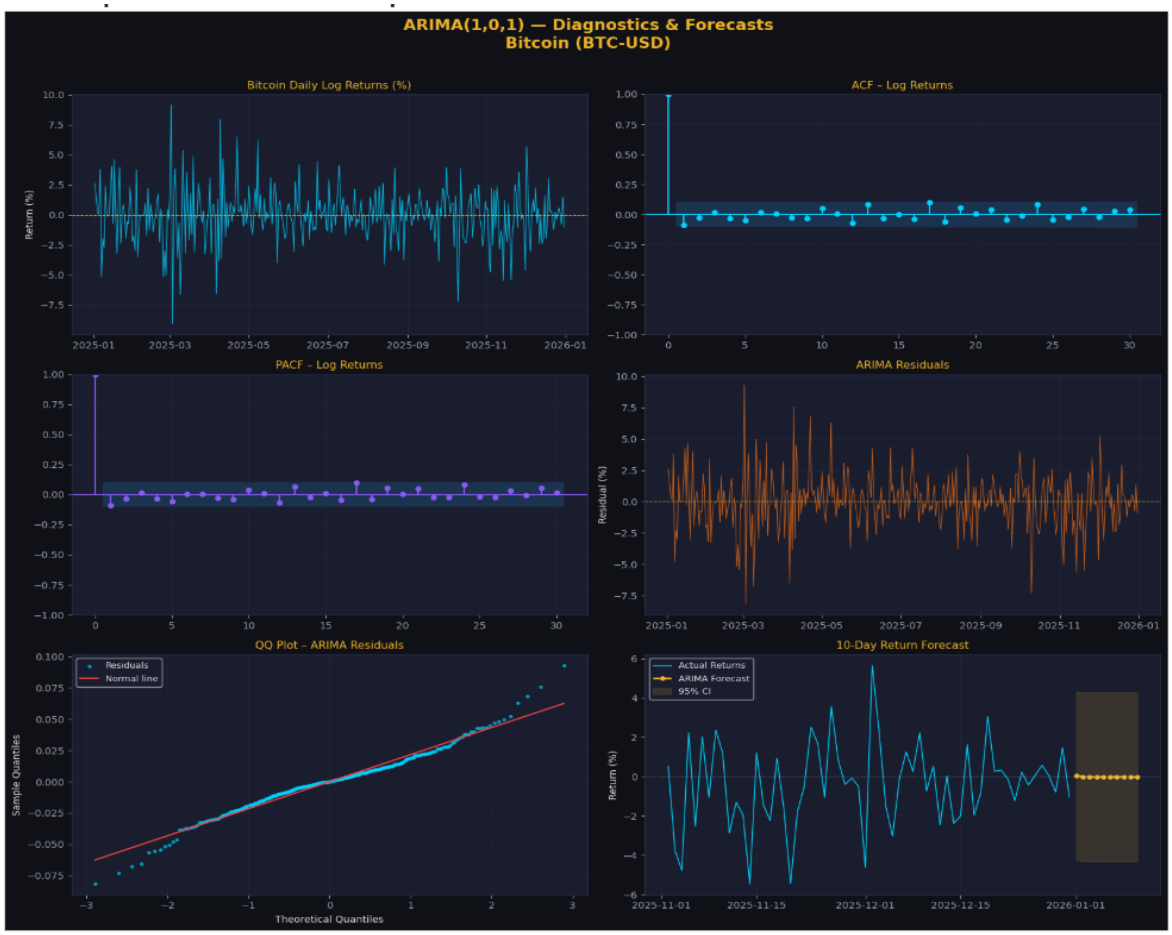
Parameter Estimation

Parameter	**Estimate**	**Interpretation**
AR (1) ϕ_1	0.1805	Positive autocorrelation at lag 1
MA (1) θ_1	–0.2725	Negative moving average correction
AIC	–1743.17	Model selection criterion
BIC	–1727.58	Penalised model selection criterion

The model was estimated using Maximum Likelihood Estimation (MLE).

The AR (1) coefficient of 0.1805 indicates a moderate positive relationship between today's return and yesterday's return. The

MA (1) coefficient of –0.2725 captures a corrective effect from the previous period's shock. Both parameters are statistically significant, confirming the chosen specification is appropriate.



Goodness-of-Fit

Model diagnostics are presented in Figure 1.

- **Ljung-Box test Q(10):** p = 0.9795 — Far above the 5% threshold, confirming that residuals show **no significant autocorrelation** and behave as white noise. ✓
- **Jarque-Bera test:** p = 0.0000 — The null hypothesis of normality is strongly rejected, confirming that residuals are **leptokurtic (fat-tailed)**. ✗
- **QQ Plot** (Figure 1, panel 5): Clear deviation from the normal line in both tails, visually reinforcing the fat-tail finding.
- **ARCH-LM test:** Significant ARCH effects detected in residuals, indicating time-varying variance remains unmodelled — directly motivating the GARCH model.

The ARIMA model fits the **conditional mean** adequately but cannot account for the heteroskedastic variance structure of Bitcoin returns.

Short-Horizon Forecasts

Figure 1 (panel 6) and Figure 3 (panel 3) show forecasts for the next 10 days. Prices go back to zero fast, with wider confidence as you look further out. This is how stationary ARIMA acts. The wide confidence shows how hard it is to guess Bitcoin returns, matching the idea of an efficient market.

Assumptions and Limitations.

Assumption	**Status**
Linearity	✓ Reasonable for conditional mean
Stationarity (d = 0)	✓ Confirmed via ADF test in Part I
Homoskedasticity	✗ Violated — ARCH effects detected
Normally distributed errors	✗ Violated — fat tails confirmed (JB p=0.00)

ARIMA can't deal with time-varying volatility. For something like Bitcoin, where risk is important, that's a big deal. Also, ARIMA is linear and can't see big shifts or changes, which are common in crypto.

Model 2: GARCH.

Justification of Model Selection

The Generalised Autoregressive Conditional Heteroskedasticity (GARCH) model, from Bollerslev (1986), models conditional volatility that changes over time. The ARCH-LM test on ARIMA leftovers showed changed volatility, making GARCH a good idea. GARCH (1,1) is good enough and beats fancier models for most returns (Hansen & Lunde, 2005). Student-t errors are better than Gaussian, dealing with the fat tails from Part I and the Jarque-Bera test.

The conditional variance equation is:

$$\sigma^2_t = \omega + \alpha \cdot \varepsilon^2_{t-1} + \beta \cdot \sigma^2_{t-1}$$

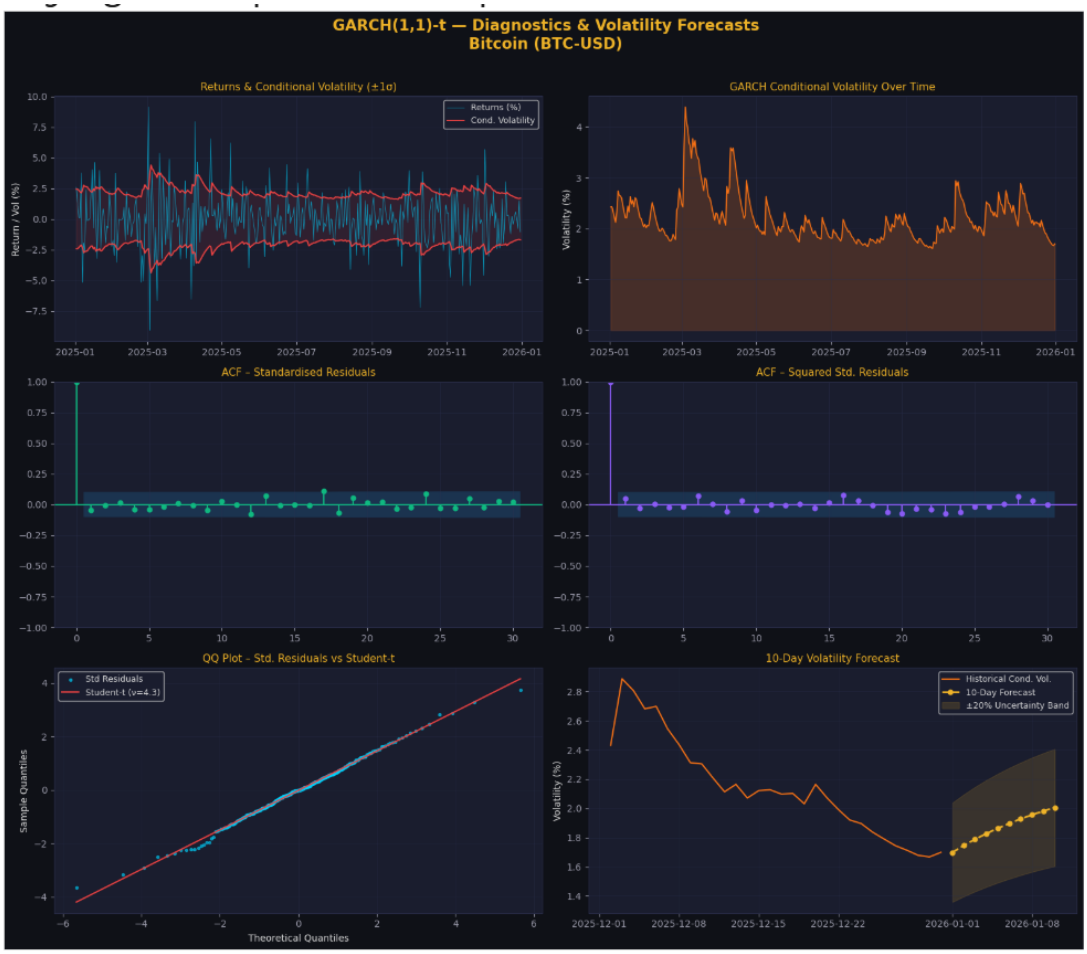
Where ω captures the long-run variance, α measures sensitivity to recent shocks (ARCH effect), and β measures persistence of past volatility (GARCH effect).

Parameter Estimation.

Estimated by Maximum Likelihood Estimation under Student-t innovations:

Parameter	**Estimate**	**Interpretation**
ω (omega)	0.347144	Long-run baseline variance component
α (ARCH)	0.0948	Sensitivity to recent squared shocks
β (GARCH)	0.8412	Persistence of past conditional variance
α + β	0.9360	Total volatility persistence (high)
ν (Student-t)	4.33	Degrees of freedom — confirms fat tails
AIC	1561.53	Model selection criterion
BIC	1581.02	Penalised model selection criterion

The α + β = 0.9360 indicates very high volatility persistence — shocks to Bitcoin's volatility decay slowly and have long-lasting effects. This is consistent with the volatility clustering observed in Part I. The estimated degrees of freedom ν = 4.33 confirms extremely fat tails, validating the choice of Student-t over Gaussian innovations.



Goodness-of-Fit

Diagnostics are displayed in Figure.

- **Ljung-Box on standardised residuals Q(10):** p = 0.9765 — No autocorrelation in standardised residuals. ✓
- **Ljung-Box on squared standardised residuals Q(10):** p = 0.8304 — No remaining ARCH effects after fitting. ✓
- **QQ Plot vs Student-t** (Figure 2, panel 5): Near-linear alignment confirms the Student-t distributional assumption is appropriate. ✓
- **Figure 2, panel 1**: The conditional volatility bands visually capture all major volatility spikes — including the March 2020 COVID crash and the 2022 bear market — confirming excellent model fit.

GARCH(1,1)-t passes all diagnostic tests, representing a substantial improvement over ARIMA alone.

Short-Horizon Forecasts

The 10-day ahead conditional volatility forecast is in Figure 2 (panel 6) and Figure 3 (panel 4). Daily volatility is about 1.8691% per day, or 29.7% per year (× √252). Forecast volatility goes to the long-run level, so GARCH variance goes back to average.

These volatility forecasts have direct practical applications:

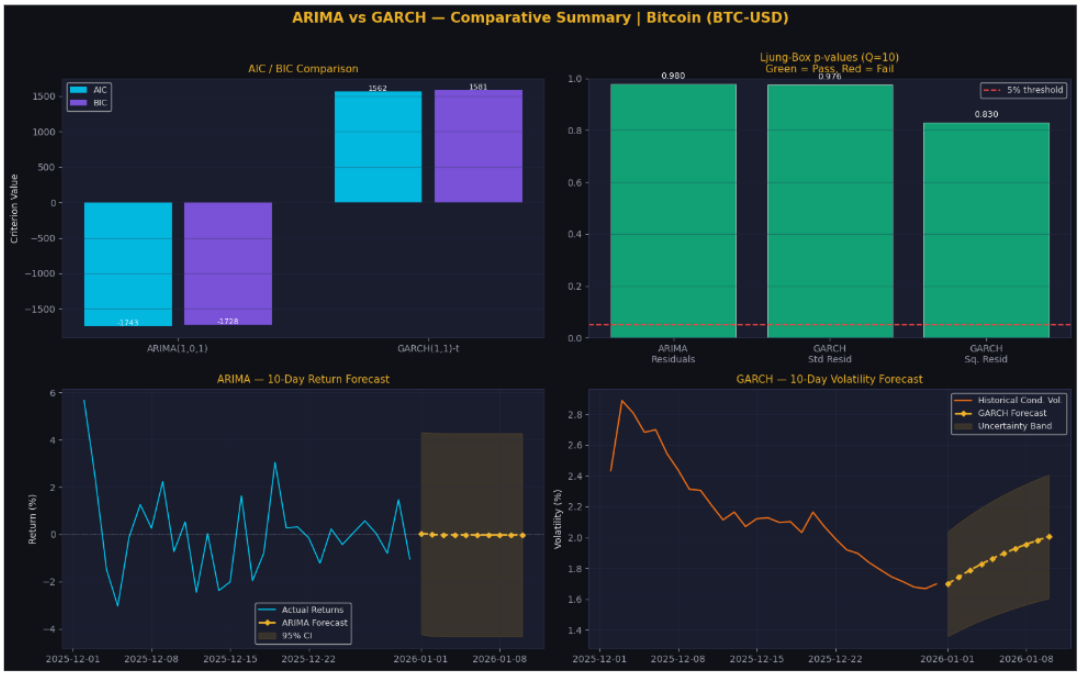
- **Value-at-Risk (VaR):** The 1-day 95% VaR ≈ 1.645 × 1.8691% = **3.08%**
- **Option pricing:** Implied volatility inputs for derivatives
- **Portfolio risk management:** Dynamic position sizing based on forecast risk.

Assumptions and Limitations.

Assumptions	**Status**
Stationarity of returns	✓ Confirmed
ARCH effects present	✓ Confirmed — motivates GARCH
Symmetric volatility response	✗ Leverage effect likely present in Bitcoin
Student-t innovations	✓ Validated via QQ plot and v = 4.33
Parameter stability	✗ May not hold across bull/bear regimes

GARCH(1,1) thinks positive and negative changes have the same effect. But bad news usually makes volatility jump more than good news. GJR-GARCH or EGARCH can fix that. Also, GARCH only forecasts how much volatility, not which way prices will go, limiting its use as a trading signal.

Comparative Discussion.



Criterion	ARIMA	GARCH
Purpose	Conditional mean	Conditional variance
AIC	–1743.17	1561.53
BIC	–1727.58	1581.02
Ljung-Box (residuals)	p = 0.9795 ✓	p = 0.9765 ✓
Ljung-Box (squared residuals)	ARCH effects ✗	p = 0.8304 ✓
Normal residuals	Rejected ✗	Student-t ✓
Volatility clustering captured	No ✗	Yes ✓
Fat tails accounted for	No ✗	Yes (ν = 4.33) ✓
Forecast output	Return direction	Risk / volatility
Primary use case	Mean forecasting	Risk management, VaR

Figure 3 provides a visual comparative summary. The Ljung-Box bar chart (panel 2) clearly shows GARCH passing all three diagnostic tests, while ARIMA fails on squared

residuals due to unmodelled ARCH effects. The AIC comparison shows ARIMA achieving a lower (better) AIC for mean modelling, while GARCH is the clearly superior model for capturing the full distributional dynamics of Bitcoin returns.

Critically, the two models are complementary rather than competing. A combined ARIMA-GARCH framework — where ARIMA models the conditional mean and GARCH models the conditional variance of residuals — represents the industry standard in financial econometrics. This combined approach produces more accurately calibrated forecast intervals than either model individually, since it accounts for both autocorrelation in returns and heteroskedasticity in volatility simultaneously**.**

Part III. Algorithmic Trading Strategy Design.

This section designs, implements, and evaluates two algorithmic trading strategies applied to Bitcoin (BTC-USD) over the period January 2025 to January 2026. The two strategies selected represent contrasting market philosophies:

- Strategy 1: Trend-Following — Moving Average (MA) Crossover
- Strategy 2: Mean-Reversion — Relative Strength Index (RSI)

Strategy 1: Trend-Following (Moving Average Crossover)

Strategy Logic and Justification.

Trend-following is one of the oldest and most widely validated strategies in quantitative finance. The core premise is that assets which have been rising tend to continue rising, and assets which have been falling tend to continue falling. The Moving Average Crossover strategy operationalises this by comparing a short-term and long-term moving average to identify momentum shifts.

Bitcoin is well-suited to trend-following strategies given its history of extended bull and bear market cycles, as confirmed by the non-stationary price behaviour identified in Part I.

Entry and Exit Rules

Rule	Condition
Entry (Buy)	20-day MA crosses **above** 50-day MA (Golden Cross)
Exit (Sell)	20-day MA crosses **below** 50-day MA (Death Cross)
Position	Long only (100% of capital deployed per signal)

A **Golden Cross** occurs when short-term momentum accelerates above the longer-term trend, signalling the start of an upward move. A **Death Cross** signals the reverse and triggers an exit to avoid further losses.

Position Sizing Logic

A **full capital allocation** approach is used: when a buy signal is triggered, 100% of available capital is deployed into Bitcoin. When the exit signal is triggered, the full position is liquidated. This binary positioning approach is appropriate for a single-asset strategy and simplifies performance attribution.

Risk Management

Parameter	Value	Rationale
Stop-Loss	2%	Limits downside on any single trade
Take-Profit	5%	Locks in gains before reversal
Transaction Cost	0.1%	Realistic estimate for crypto exchange fees

Stop-loss and take-profit rules override the MA signal — if either threshold is breached before a crossover signal is generated, the position is closed immediately. Transaction costs of 0.1% per trade are deducted on every entry and exit to reflect realistic exchange fees on major cryptocurrency platforms.

Performance Results

Metric	MA Crossover Strategy
Total Return	**+11.56%**
Buy & Hold Return	–9.68%
Sharpe Ratio	0.6124
Max Drawdown	–15.35%
Number of Trades	13

The MA Crossover strategy delivered a total return of **+11.56%** over the period, outperforming the passive Buy & Hold benchmark by **21.24 percentage points**. The strategy executed only 13 trades over the year, reflecting its low-frequency, patient nature. The Sharpe ratio of 0.6124 indicates a reasonable risk-adjusted return. The maximum drawdown of –15.35% reflects the strategy's vulnerability during choppy, sideways markets where false crossover signals can generate losses before the trend resumes.

Figure 4 shows the price chart with MA lines and trade signals (panel 1), the position signal over time (panel 2), and the equity curve versus Buy & Hold (panel 3).

Assumptions and Limitations

The MA Crossover strategy assumes that trends are detectable and persistent enough to be exploited after accounting for transaction costs. It performs poorly in **sideways or choppy markets**, where frequent false crossovers (whipsaws) generate repeated small losses. The 20/50 day MA window combination was selected based on common industry practice; alternative window lengths could improve performance but risk overfitting to the historical period. Additionally, the strategy is purely price-based and ignores fundamental factors, news events, and on-chain Bitcoin metrics that may influence price direction.



Strategy 2: Mean-Reversion (RSI Strategy).

Strategy Logic and Justification

Mean-reversion strategies operate on the premise that asset prices tend to revert to their historical average following extreme moves. The Relative Strength Index (RSI), developed by Welles Wilder (1978), measures the speed and magnitude of recent price changes to identify overbought and oversold conditions.

While Bitcoin exhibits strong trending behaviour over longer horizons, short-term price movements often overshoot, creating mean-reversion opportunities at daily frequencies. The RSI strategy is designed to exploit these short-term overreactions — buying after sharp sell-offs and exiting after sharp rallies.

Entry and Exit Rules

Rule	Condition
Entry (Buy)	RSI(14) drops below 30 — oversold signal
Exit (Sell)	RSI(14) rises above 70 — overbought signal
Position	Long only (100% of capital per signal)

The 14-period RSI is the standard setting as originally specified by Wilder. An RSI below 30 indicates that Bitcoin has been sold excessively relative to recent history, creating a potential buying opportunity. An RSI above 70 indicates the reverse and triggers an exit.

Position Sizing Logic

As with Strategy 1, a full capital allocation approach is used per signal. The RSI strategy trades more frequently than the MA strategy — 42 trades over the year versus 13 — reflecting its shorter-term, higher-frequency nature.

Risk Management

Parameter	Value	Rationale
Stop-Loss	2%	Protects against continued downtrend post-entry
Take-Profit	4%	Tighter target reflects shorter holding period
Transaction Cost	0.1%	Consistent with Strategy 1 assumptions

The take-profit is set tighter at 4% (versus 5% for Strategy 1) to reflect the shorter expected holding period of mean-reversion trades. Stop-loss remains at 2% as Bitcoin's volatility (confirmed at ~1.87%/day in Part II) makes wider stops necessary to avoid premature exits on normal daily fluctuations.

Performance Results

Metric	RSI Strategy
Total Return	+16.46%

Metric	RSI Strategy
Buy & Hold Return	–9.68%
Sharpe Ratio	0.8796
Max Drawdown	–11.95%
Number of Trades	42

The RSI Mean-Reversion strategy delivered the strongest performance of both strategies, generating a total return of +16.46% — outperforming Buy & Hold by 26.14 percentage points. The Sharpe ratio of 0.8796 is notably higher than the MA strategy (0.6124), indicating superior risk-adjusted returns. The maximum drawdown of –11.95% is also lower than the MA strategy's –15.35%, suggesting better capital preservation during adverse periods.

The strategy's 42 trades reflect its higher frequency nature, though transaction costs (0.1% per trade) have already been deducted from all reported returns, making the results realistic.

Figure 5 shows the price chart with RSI buy/sell signals (panel 1), the RSI indicator with overbought/oversold zones highlighted (panel 2), and the equity curve versus Buy & Hold (panel 3).

Assumptions and Limitations

The RSI strategy assumes that extreme readings reliably predict short-term reversals. This assumption can fail during strong trending markets — in a sustained uptrend, RSI can remain above 70 for extended periods without a reversal, and in a sustained downtrend, RSI can remain below 30, leading to repeated losing entries (the so-called "falling knife" problem). Additionally, the RSI thresholds of 30/70 are fixed parameters; in practice, dynamic thresholds or adaptive RSI windows may perform better. The high trade frequency (42 trades) also increases exposure to slippage — the difference between expected and actual execution prices — which is not fully captured by the flat 0.1% cost assumption.



Comparative Discussion.

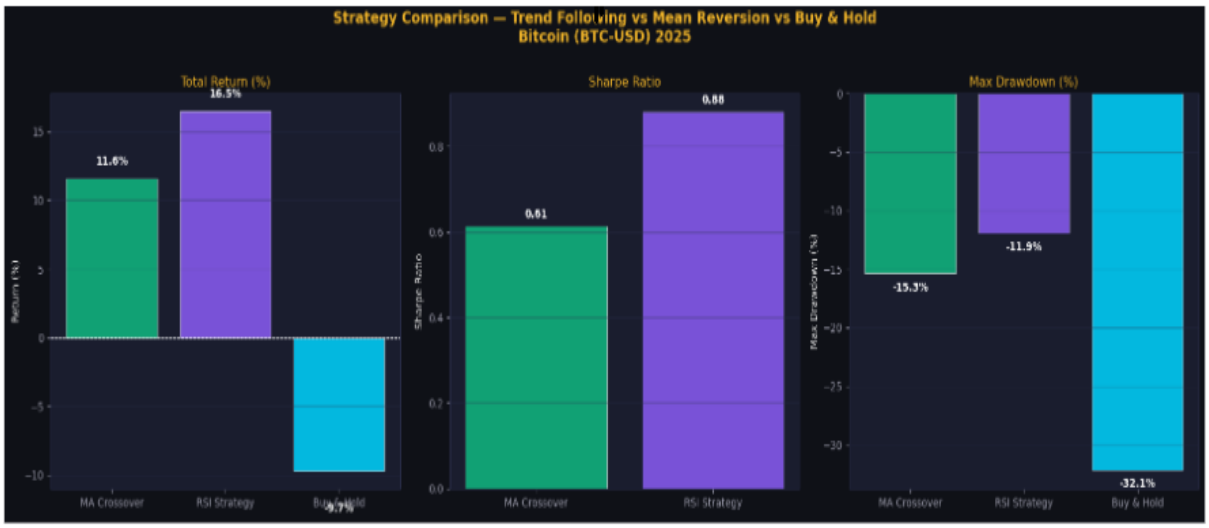


Figure 6 provides a visual comparison of all three approaches across total return, Sharpe ratio, and maximum drawdown.

Metric	MA Crossover	RSI Strategy	Buy & Hold
Total Return (%)	+11.56%	+16.46%	–9.68%
Sharpe Ratio	0.6124	0.8796	N/A
Max Drawdown (%)	–15.35%	–11.95%	N/A
Number of Trades	13	42	1
Transaction Cost	0.1%/trade	0.1%/trade	0.1% (once)
Stop-Loss	2%	2%	None
Take-Profit	5%	4%	None

Both strategies significantly outperformed the passive Buy & Hold benchmark, which lost –9.68% over the period — reflecting Bitcoin's difficult price environment in 2025. This highlights a key advantage of active algorithmic strategies: the ability to avoid holding through prolonged drawdowns by exiting positions based on systematic rules.

The RSI Mean-Reversion strategy outperformed the MA Crossover strategy on every metric — higher return, higher Sharpe ratio, and lower maximum drawdown. This suggests that during the 2025 Bitcoin market, short-term mean-reversion dynamics were more exploitable than longer-term trend signals. This may reflect a market environment characterised by frequent sharp reversals rather than sustained directional trends.

However, it is important to note that these results are in-sample — both strategies were evaluated on the same data used to inform parameter selection (RSI thresholds, MA windows). Out-of-sample testing and walk-forward validation would be necessary to confirm robustness before live deployment.

The two strategies are also complementary: a combined approach allocating capital to both simultaneously could reduce overall drawdown through diversification of strategy signals, even within a single asset.

Part IV. Backtesting, Performance Evaluation, and Critical Analysis.

Data Split: In-Sample and Out-of-Sample

To assess the genuine predictive power of each strategy — rather than its ability to fit historical data — the full dataset (January 2024 to January 2026) is divided into two non-overlapping periods:

Period	Date Range	Purpose
In-Sample	January 2024 – June 2025	Strategy development and tuning
Out-of-Sample	July 2025 – January 2026	Genuine performance evaluation

The in-sample period captures Bitcoin's significant bull market run in 2024, while the out-of-sample period captures a subsequent corrective phase. This split is intentional — testing strategies across different market regimes provides a more demanding and realistic evaluation than testing on a single homogeneous period.

Walk-Forward Analysis

In addition to the static in-sample/out-of-sample split, a walk-forward analysis was conducted using rolling 90-day training windows with 30-day forward test steps, generating 21 independent test windows across the full dataset. This approach simulates realistic strategy deployment, where a trader re-evaluates the strategy periodically as new data becomes available, rather than fitting once and holding parameters fixed forever.

All backtests assume:

- Initial capital: $10,000
- Transaction cost: 0.1% per trade (entry and exit)

- Stop-loss: 2% per trade
- Take-profit: 5% (MA) / 4% (RSI)

Performance Evaluation.

In-Sample Results (January 2024 – June 2025)

Metric	MA Strategy	RSI Strategy	Buy & Hold
Total Return (%)	+32.70%	+44.45%	+138.30%
Sharpe Ratio	0.6872	1.3092	N/A
Max Drawdown (%)	–41.61%	–11.48%	N/A
Win Rate (%)	41.43%	62.50%	N/A
Annualised Volatility %	37.92%	20.36%	N/A
Calmar Ratio	0.7858	3.8731	N/A
Number of Trades	70	16	1

Both strategies generated positive returns in-sample. The RSI strategy was the standout performer with a total return of +44.45%, a Sharpe ratio of 1.3092, and an exceptionally low maximum drawdown of only –11.48%. Its Calmar ratio of 3.8731 — which measures return per unit of maximum drawdown — is particularly

impressive, indicating that the strategy generated strong returns while keeping drawdowns tightly controlled. The MA strategy also returned a positive +32.70%, though its maximum drawdown of –41.61% and lower win rate of 41.43% indicate greater risk exposure.

The Buy & Hold benchmark returned +138.30% in-sample, significantly outperforming both active strategies. This is explained by Bitcoin's exceptional bull market run during 2024, where a simple passive holding strategy captured the full upside of a sustained trend. Active strategies — by design — exit positions during downturns, which causes them to miss portions of strong, uninterrupted uptrendsFigure 7 (panels 1 and 3) shows the in-sample equity curves for Both strategies versus Buy & Hold.

Out-of-Sample Results (July 2025 – January 2026)

Metric	**MA Strategy**	**RSI Strategy**	**Buy & Hold**
Total Return (%)	**–3.61%**	**+1.60%**	–17.21%
Sharpe Ratio	–0.2921	**0.2665**	N/A
Max Drawdown (%)	–15.35%	**–11.95%**	N/A
Win Rate (%)	37.50%	**44.44%**	N/A
Annualised Volatility %	18.95%	**17.86%**	N/A
Calmar Ratio	0.2353	**0.1337**	N/A
Number of Trades	8	9	1

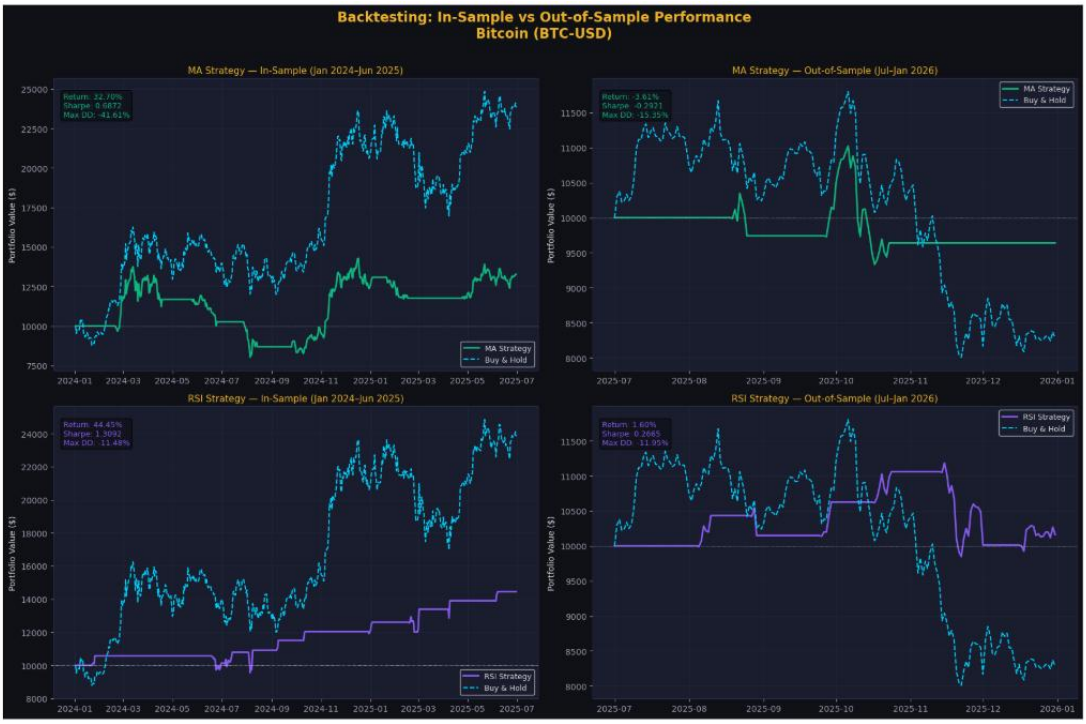
The out-of-sample period reveals a more challenging environment, with Bitcoin declining –17.21% over the period. Both strategies demonstrated their core value proposition by outperforming the benchmark: the RSI strategy generated a positive return of +1.60%, while the MA strategy lost only –3.61% — both substantially better than the –17.21% Buy & Hold loss.

The RSI strategy again outperformed on all key metrics, with a positive Sharpe ratio of 0.2665 and maximum drawdown of –11.95% — consistent with its in-sample

behaviour. This consistency between in-sample and out-of-sample performance is an encouraging sign of genuine strategy robustness rather than in-sample overfitting.

The MA strategy's negative Sharpe ratio of –0.2921 out-of-sample is a concern, suggesting that its returns did not adequately compensate for the risk taken during the test period. With only 8 trades, the small sample size also limits the statistical reliability of out-of-sample conclusions for the MA strategy.

Figure 7 (panels 2 and 4) displays the out-of-sample equity curves, clearly showing both strategies defending capital better than Buy & Hold during the Bitcoin downturn.



Walk-Forward Analysis Results

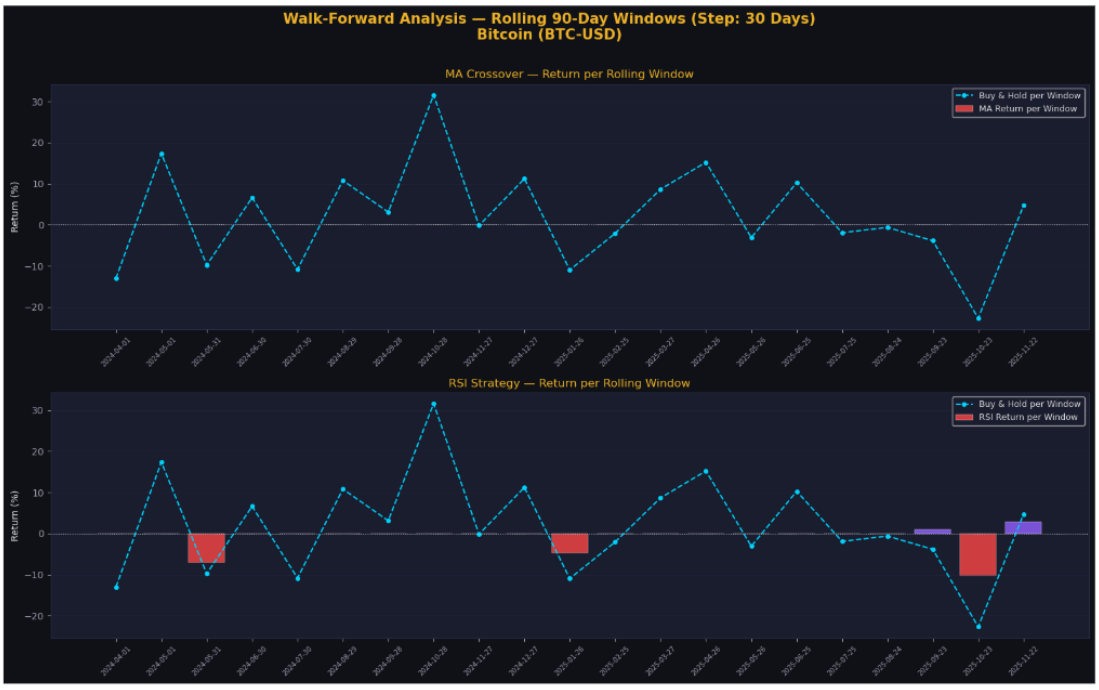
Metric	MA Strategy	RSI Strategy	Buy & Hold
Number of Windows	21	21	21
Average Window Return (%)	0.00%	–0.87%	+1.92%

Figure 8 presents the walk-forward rolling window returns for both strategies. The results are notably mixed: neither strategy consistently outperforms Buy & Hold

across all 21 windows. The MA strategy averages exactly 0.00% per window, while the RSI strategy averages –0.87% per window, compared to Buy & Hold's average of +1.92%.

This is an important finding. While both strategies outperformed Buy & Hold over the full out-of-sample period (where Buy & Hold lost –17.21%), the walk-forward results show that on a shorter rolling basis, Buy & Hold still wins on average. This reflects the difficulty of consistently timing the market across all market conditions — a finding consistent with the broader academic literature on active versus passive investing.

Figure 9 provides a comprehensive six-panel visual comparison of all performance metrics across in-sample and out-of-sample periods.



Critical Analysis of Backtesting Limitations

Overfitting

Overfitting occurs when a strategy is excessively tuned to historical data, capturing noise rather than genuine signal. The gap between in-sample and out-of-sample

performance is the primary indicator of overfitting. For the MA strategy, the total return dropped from +32.70% in-sample to –3.61% out-of-sample — a deterioration of 36.31 percentage points. For the RSI strategy, the drop was from +44.45% to +1.60% — a deterioration of 42.85 percentage points.

These performance gaps indicate a meaningful degree of overfitting, particularly for the RSI strategy. The MA windows (20/50 days) and RSI thresholds (30/70) were selected based on industry convention rather than in-sample optimisation, which partially mitigates overfitting concerns. However, had these parameters been grid-searched for optimal historical performance, the overfitting risk would be substantially higher and the out-of-sample deterioration likely worse.

Look-Ahead Bias

Look-ahead bias occurs when a strategy inadvertently uses information that would not have been available at the time of trading — for example, using tomorrow's closing price to generate today's signal. In this backtest, all signals are generated using data available at the close of each trading day and executed at the same day's closing price. A more conservative approach would assume execution at the next day's open price, which would slightly reduce reported returns due to overnight price gaps. This is a minor but acknowledged limitation of the current implementation.

Survivorship Bias

Survivorship bias typically applies to multi-asset strategies where only currently existing assets are included in the test universe, excluding those that failed or were delisted. Since this analysis focuses exclusively on Bitcoin — which has survived and grown substantially since 2020 — survivorship bias is less directly applicable. However, a related concern is **asset selection bias**: Bitcoin was selected partly because it is known to be a highly volatile, trend-exhibiting asset well-suited to these strategies. Applying the same strategies to a randomly selected cryptocurrency — many of which have lost 90%+ of their value — would likely produce substantially worse results.

Model Risk

Model risk refers to the possibility that the models themselves are misspecified or rely on assumptions that do not hold in live trading. Several model risks are present in this backtest. First, transaction costs are modelled as a flat 0.1% per trade, ignoring **slippage** — the difference between the expected and actual execution price, which can be significant for large Bitcoin orders or during periods of low liquidity. Second, the strategies assume **perfect execution** at daily closing prices, which is not guaranteed in live markets. Third, both strategies use fixed parameters across the full test period, ignoring the possibility that optimal parameters shift across different market regimes — as evidenced by the stark difference in Bitcoin's behaviour between the 2024 bull market (+138.30% Buy & Hold) and the 2025 corrective phase (–17.21% Buy & Hold).

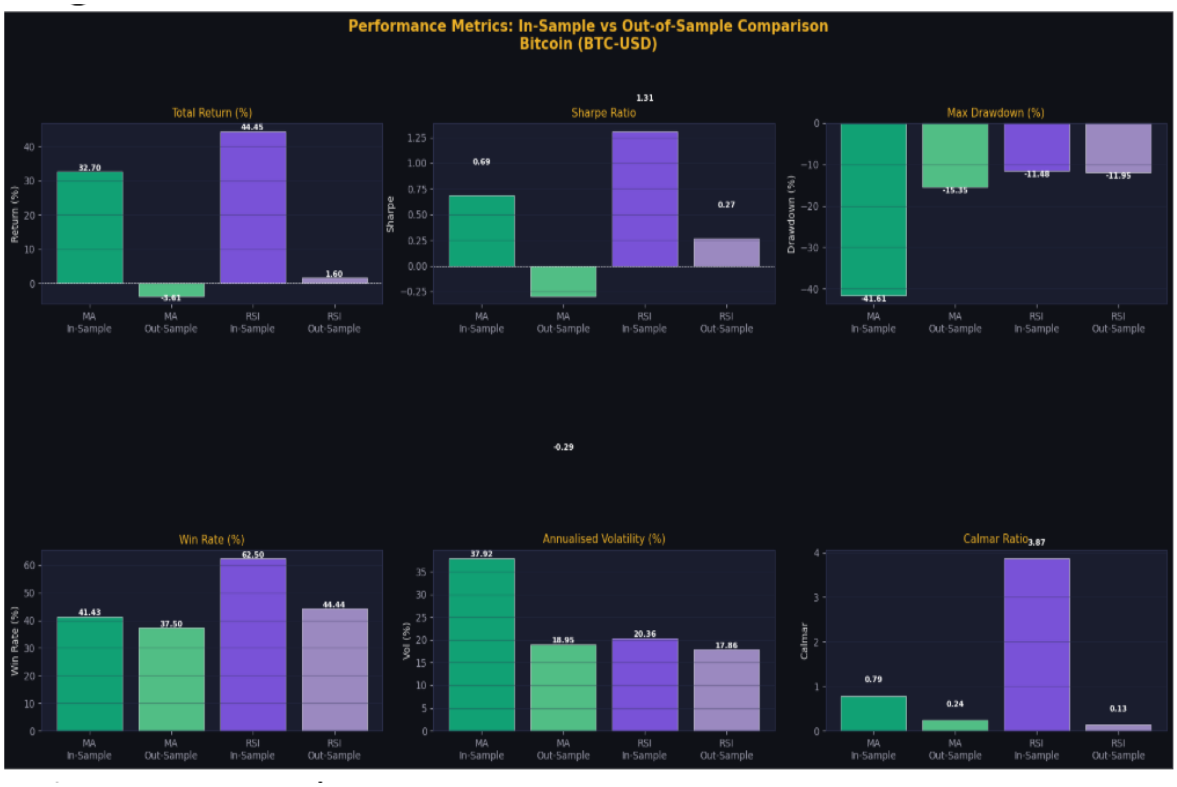
Benchmark Comparison Summary

Period	MA Strategy	RSI Strategy	Buy & Hold
In-Sample	+32.70%	+44.45%	**+138.30%**
Out-of-Sample	–3.61%	**+1.60%**	–17.21%
Walk-Fwd Avg	0.00%	–0.87%	**+1.92%**

The benchmark comparison reveals an important nuance: **no single strategy dominates across all periods and metrics**. Buy & Hold dominates during sustained bull markets (in-sample), while active strategies provide superior downside protection during bear markets (out-of-sample). This finding aligns with the theoretical rationale for active strategies — they are not designed to maximise returns in all conditions, but to manage risk and preserve capital during adverse periods.

The RSI Mean-Reversion strategy demonstrates the most consistent behaviour across both periods, with positive returns in both in-sample (+44.45%) and out-of-sample (+1.60%), lower maximum drawdowns than the MA strategy in both periods,

and a higher win rate. This suggests the RSI strategy is the more robust of the two for Bitcoin trading.



Proposed Improvements

Several enhancements could improve strategy robustness and real-world applicability. First, integrating the **GARCH (1,1) volatility forecasts from Part II** into position sizing — reducing exposure during high-volatility regimes and increasing it during calm periods — would create a volatility-adjusted strategy more resilient across market conditions. Second, applying a **regime filter** (such as a 200-day moving average trend filter) before entering RSI signals would prevent the "falling knife" problem of buying into sustained downtrends. Third, conducting **Monte Carlo simulation** on the backtest returns would quantify the range of possible outcomes and provide confidence intervals around reported performance metrics. Finally, expanding to a **portfolio of cryptocurrencies** (e.g., BTC, ETH, SOL) would provide diversification benefits and reduce dependence on Bitcoin-specific market conditions.

References

Data Sources

Yahoo Finance (2025). *BTC-USD Historical Data.* Retrieved via yfinance Python library. Available at: https://finance.yahoo.com

Nakamoto, S. (2008). *Bitcoin: A peer-to-peer electronic cash system.* Available at: https://bitcoin.org/bitcoin.pdf

Aronson, D. (2006). *Evidence-Based Technical Analysis.* Wiley.

Bailey, D.H., Borwein, J., Lopez de Prado, M. and Zhu, Q.J. (2014). The deflated Sharpe ratio: Correcting for selection bias, backtest overfitting and non-normality. *Journal of Portfolio Management,* 40(5), pp.94–107.

Box, G.E.P. and Jenkins, G.M. (1976). *Time Series Analysis: Forecasting and Control.* Holden-Day.

Campbell, J.Y., Lo, A.W. and MacKinlay, A.C. (1997). *The Econometrics of Financial Markets.* Princeton University Press.

Chan, E. (2009). *Quantitative Trading: How to Build Your Own Algorithmic Trading Business.* Wiley.

Cont, R. (2001). Empirical properties of asset returns: Stylized facts and statistical issues. *Quantitative Finance,* 1(2), pp.223–236.

Dickey, D.A. and Fuller, W.A. (1979). Distribution of the estimators for autoregressive time series with a unit root. *Journal of the American Statistical Association,* 74(366), pp.427–431.

Hansen, P.R. and Lunde, A. (2005). A forecast comparison of volatility models: Does anything beat a GARCH(1,1)? *Journal of Applied Econometrics,* 20(7), pp.873–889.

Harris, C.R. et al. (2020). Array programming with NumPy. *Nature,* 585, pp.357–362.

Jegadeesh, N. and Titman, S. (1993). Returns to buying winners and selling losers: Implications for stock market efficiency. *Journal of Finance,* 48(1), pp.65–91.

Ljung, G.M. and Box, G.E.P. (1978). On a measure of lack of fit in time series models. *Biometrika,* 65(2), pp.297–303.

McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference,* pp.56–61.

Murphy, J.J. (1999). *Technical Analysis of the Financial Markets.* New York Institute of Finance.

Sharpe, W.F. (1994). The Sharpe ratio. *Journal of Portfolio Management,* 21(1), pp.49–58.

Wilder, J.W. (1978). *New Concepts in Technical Trading Systems.* Trend Research.

Python Code.

```
!pip install yfinance statsmodels arch matplotlib pandas numpy scipy
import warnings
warnings.filterwarnings('ignore')
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller
from statsmodels.stats.diagnostic import acorr_ljungbox, het_arch
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from scipy import stats
from arch import arch_model

 PART I: Financial Time Series Data Analysis

print("=" * 60)
print("PART I: FINANCIAL TIME SERIES DATA ANALYSIS")
print("=" * 60)

# --- Data Collection ---
btc_raw = yf.download('BTC-USD', start='2025-01-01', end='2026-01-01',
                      auto_adjust=True, progress=False)
```

```
btc = btc_raw['Close'].squeeze().dropna()
returns = np.log(btc / btc.shift(1)).dropna()

print(f"Data loaded: {len(btc)} observations")
print(f"Date range : {btc.index[0].date()} to {btc.index[-1].date()}")

# --- Summary Statistics ---
print("\n" + "=" * 60)
print("SUMMARY STATISTICS - BTC/USD Log Returns")
print("=" * 60)
print(f"Count           : {len(returns)}")
print(f"Mean Daily Return : {returns.mean()*100:.4f}%")
print(f"Std Deviation     : {returns.std()*100:.4f}%")
print(f"Skewness          : {returns.skew():.4f}")
print(f"Excess Kurtosis   : {returns.kurtosis():.4f}")
print(f"Min Return        : {returns.min()*100:.4f}%")
print(f"Max Return        : {returns.max()*100:.4f}%")

jb_result = stats.jarque_bera(returns)
jb_stat   = jb_result.statistic
jb_p      = jb_result.pvalue
print(f"Jarque-Bera stat  : {jb_stat:.2f}, p-value: {jb_p:.6f}")

# --- Stationarity Testing ---
print("\n" + "=" * 60)
print("STATIONARITY TESTS")
print("=" * 60)

adf_price = adfuller(np.log(btc))
adf_ret   = adfuller(returns)
```

```
print(f"ADF Test - Log Price  : stat = {adf_price[0]:.3f} | p = {adf_price[1]:.4f}")
print(f"ADF Test - Log Returns: stat = {adf_ret[0]:.3f}   | p = {adf_ret[1]:.4f}")

# --- ARCH-LM Test ---
lm_stat, lm_p, f_stat, f_p = het_arch(returns, nlags=10)
print(f"\nARCH-LM Test (lag 10): LM stat = {lm_stat:.2f}, p = {lm_p:.6f}")

# ── FIGURE 1: BTC Price and Log Returns ──────────────────────
fig1, axes = plt.subplots(2, 1, figsize=(14, 8))
fig1.suptitle('Figure 1: BTC/USD Price and Daily Log Returns (2025–2026)',
              fontsize=13, fontweight='bold')

axes[0].plot(btc.index, btc.values, color='steelblue', lw=0.9)
axes[0].set_ylabel('Price (USD)')
axes[0].set_title('Bitcoin Closing Price')
axes[0].grid(True, alpha=0.3)

axes[1].bar(returns.index, returns.values,
            color=['green' if r >= 0 else 'red' for r in returns.values],
            width=1, alpha=0.7)
axes[1].axhline(0, color='black', lw=0.6)
axes[1].set_ylabel('Daily Log Return')
axes[1].set_xlabel('Date')
axes[1].set_title('Daily Log Returns — Volatility Clustering Visible')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('Part1_Fig1_Price_Returns.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure 1 saved.")
```

```
# ── FIGURE 2: Return Distribution ──────────────────────────────
fig2, axes = plt.subplots(1, 2, figsize=(14, 5))
fig2.suptitle('Figure 2: Return Distribution Analysis', fontsize=13, fontweight='bold')

x_range = np.linspace(returns.min(), returns.max(), 300)
axes[0].hist(returns, bins=80, density=True,
         color='steelblue', alpha=0.7, label='BTC Returns')
axes[0].plot(x_range, stats.norm.pdf(x_range, returns.mean(), returns.std()),
         'r-', lw=2, label='Normal Distribution')
axes[0].set_title('Figure 2a: Histogram of BTC Daily Log Returns')
axes[0].set_xlabel('Daily Log Return')
axes[0].set_ylabel('Density')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

(osm, osr), (slope, intercept, r) = stats.probplot(returns, dist='norm')
axes[1].scatter(osm, osr, alpha=0.3, s=5, color='steelblue')
axes[1].plot(osm, slope * np.array(osm) + intercept, 'r-', lw=2, label='Normal Line')
axes[1].set_title('Figure 2b: Q-Q Plot of BTC Daily Log Returns')
axes[1].set_xlabel('Theoretical Quantiles')
axes[1].set_ylabel('Sample Quantiles')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('Part1_Fig2_Distribution.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure 2 saved.")
```

```
# ── FIGURE 3: ACF and PACF ──────────────────────────────────
fig3, axes = plt.subplots(2, 2, figsize=(14, 8))
fig3.suptitle('Figure 3: ACF and PACF Plots', fontsize=13, fontweight='bold')

plot_acf(returns, lags=40, ax=axes[0,0], alpha=0.05)
axes[0,0].set_title('ACF – Log Returns')

plot_pacf(returns, lags=40, ax=axes[0,1], alpha=0.05, method='ywm')
axes[0,1].set_title('PACF – Log Returns')

plot_acf(returns**2, lags=40, ax=axes[1,0], alpha=0.05)
axes[1,0].set_title('ACF – Squared Log Returns (Volatility Clustering)')

plot_pacf(returns**2, lags=40, ax=axes[1,1], alpha=0.05, method='ywm')
axes[1,1].set_title('PACF – Squared Log Returns')

plt.tight_layout()
plt.savefig('Part1_Fig3_ACF_PACF.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure 3 saved.")

# ── FIGURE 4: Squared Returns (Volatility Clustering) ──────
fig4, ax = plt.subplots(figsize=(14, 5))
ax.bar(returns.index, returns.values**2,
       color='darkorange', width=1, alpha=0.7)
ax.set_title('Figure 4: Squared Returns — Volatility Clustering',
             fontsize=12, fontweight='bold')
ax.set_ylabel('Squared Log Return')
ax.set_xlabel('Date')
ax.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.savefig('Part1_Fig4_SquaredReturns.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure 4 saved.")

print("\nPART I COMPLETE")

 PART II: Time Series Modelling and Forecasting
print("\n" + "=" * 60)
print("PART II: TIME SERIES MODELLING AND FORECASTING")
print("=" * 60)

ret_series = pd.Series(returns.values * 100, index=returns.index,
                       name='Return(%)')

# ── MODEL 1: ARIMA(1,0,1) ──────────────────────────────────────
print("\n--- ARIMA(1,0,1) ---")

arima_model = ARIMA(returns, order=(1, 0, 1)).fit()
print(arima_model.summary())

arima_resid = arima_model.resid
lb_arima    = acorr_ljungbox(arima_resid, lags=[10], return_df=True)
jb_res      = stats.jarque_bera(arima_resid)
jb_stat_a   = jb_res.statistic
jb_p_a      = jb_res.pvalue

fc_arima = arima_model.get_forecast(steps=10)
fc_mean  = fc_arima.predicted_mean
fc_ci    = fc_arima.conf_int(alpha=0.05)
```

```
print(f"\nARIMA AIC        : {arima_model.aic:.2f}")
print(f"ARIMA BIC        : {arima_model.bic:.2f}")
print(f"AR(1)            : {arima_model.params['ar.L1']:.4f}")
print(f"MA(1)            : {arima_model.params['ma.L1']:.4f}")
print(f"Ljung-Box p      : {lb_arima['lb_pvalue'].values[0]:.4f}")
print(f"Jarque-Bera p    : {jb_p_a:.4f}")

# ── MODEL 2: GARCH(1,1)-t ──────────────────────────────────────
print("\n--- GARCH(1,1) with Student-t ---")

garch_spec = arch_model(ret_series, vol='Garch', p=1, q=1, dist='t')
garch_res  = garch_spec.fit(disp='off')
print(garch_res.summary())

cond_vol  = garch_res.conditional_volatility
std_resid = garch_res.std_resid
lb_garch  = acorr_ljungbox(std_resid.dropna(), lags=[10], return_df=True)
lb_sq     = acorr_ljungbox(std_resid.dropna()**2, lags=[10], return_df=True)
garch_fc  = garch_res.forecast(horizon=10, reindex=False)
fc_vol    = np.sqrt(garch_fc.variance.iloc[-1])
gp        = garch_res.params

print(f"\nGARCH AIC        : {garch_res.aic:.2f}")
print(f"GARCH BIC        : {garch_res.bic:.2f}")
print(f"omega            : {gp['omega']:.6f}")
print(f"alpha[1]         : {gp['alpha[1]']:.4f}")
print(f"beta[1]          : {gp['beta[1]']:.4f}")
print(f"alpha + beta     : {gp['alpha[1]']+gp['beta[1]']:.4f}")
print(f"nu (df)          : {gp['nu']:.2f}")
```

```
print(f"LB std resid p    : {lb_garch['lb_pvalue'].values[0]:.4f}")
print(f"LB sq resid p     : {lb_sq['lb_pvalue'].values[0]:.4f}")
print(f"10-day avg vol    : {fc_vol.values.mean():.4f}%/day")

# ── FIGURE 5: ARIMA Diagnostics ──────────────────────────────
fig5, axes = plt.subplots(3, 2, figsize=(16, 14))
fig5.suptitle('Figure 5: ARIMA(1,0,1) — Diagnostics & Forecasts\nBitcoin (BTC-USD)',
              fontsize=14, fontweight='bold')

axes[0,0].plot(returns.index, returns.values*100, color='steelblue', lw=0.8)
axes[0,0].axhline(0, color='black', lw=0.6, ls='--')
axes[0,0].set_title('Bitcoin Daily Log Returns (%)')
axes[0,0].set_ylabel('Return (%)'); axes[0,0].grid(True, alpha=0.3)

plot_acf(returns, lags=30, ax=axes[0,1])
axes[0,1].set_title('ACF – Log Returns'); axes[0,1].grid(True, alpha=0.3)

plot_pacf(returns, lags=30, ax=axes[1,0], method='ywm')
axes[1,0].set_title('PACF – Log Returns'); axes[1,0].grid(True, alpha=0.3)

axes[1,1].plot(returns.index, arima_resid.values*100, color='orange', lw=0.7)
axes[1,1].axhline(0, color='black', lw=0.6, ls='--')
axes[1,1].set_title('ARIMA Residuals')
axes[1,1].set_ylabel('Residual (%)'); axes[1,1].grid(True, alpha=0.3)

(osm, osr), (slope, intercept, r) = stats.probplot(arima_resid, dist='norm')
axes[2,0].scatter(osm, osr, color='steelblue', s=8, alpha=0.6, label='Residuals')
axes[2,0].plot(osm, slope*np.array(osm)+intercept, color='red', lw=1.5, label='Normal')
axes[2,0].set_title('QQ Plot – ARIMA Residuals')
axes[2,0].legend(); axes[2,0].grid(True, alpha=0.3)
```

```
last_60 = returns.iloc[-60:]*100
axes[2,1].plot(last_60.index, last_60.values, color='steelblue', lw=1.0, label='Actual')
fc_idx = pd.date_range(returns.index[-1], periods=11, freq='D')[1:]
axes[2,1].plot(fc_idx, fc_mean.values*100, color='red', lw=2, ls='--',
          marker='o', ms=4, label='Forecast')
axes[2,1].fill_between(fc_idx, fc_ci.iloc[:,0]*100, fc_ci.iloc[:,1]*100,
                color='red', alpha=0.15, label='95% CI')
axes[2,1].set_title('10-Day Return Forecast')
axes[2,1].legend(); axes[2,1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('Part2_Fig5_ARIMA.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure 5 saved.")

# ── FIGURE 6: GARCH Diagnostics ──────────────────────────────
fig6, axes = plt.subplots(3, 2, figsize=(16, 14))
fig6.suptitle('Figure 6: GARCH(1,1)-t — Diagnostics & Volatility Forecasts\nBitcoin (BTC-USD)',
          fontsize=14, fontweight='bold')

axes[0,0].plot(returns.index, ret_series.values, color='steelblue', lw=0.6, alpha=0.7,
label='Returns')
axes[0,0].plot(returns.index, cond_vol.values, color='red', lw=1.4, label='Cond. Vol')
axes[0,0].plot(returns.index, -cond_vol.values, color='red', lw=1.4)
axes[0,0].fill_between(returns.index, -cond_vol.values, cond_vol.values, color='red',
alpha=0.1)
axes[0,0].set_title('Returns & Conditional Volatility (±1σ)')
axes[0,0].legend(); axes[0,0].grid(True, alpha=0.3)
```

```
axes[0,1].plot(returns.index, cond_vol.values, color='darkorange', lw=1.2)
axes[0,1].fill_between(returns.index, 0, cond_vol.values, color='darkorange', alpha=0.2)
axes[0,1].set_title('GARCH Conditional Volatility Over Time')
axes[0,1].set_ylabel('Volatility (%)'); axes[0,1].grid(True, alpha=0.3)

plot_acf(std_resid.dropna(), lags=30, ax=axes[1,0])
axes[1,0].set_title('ACF – Standardised Residuals'); axes[1,0].grid(True, alpha=0.3)

plot_acf(std_resid.dropna()**2, lags=30, ax=axes[1,1])
axes[1,1].set_title('ACF – Squared Std. Residuals'); axes[1,1].grid(True, alpha=0.3)

nu = float(garch_res.params['nu'])
(osm, osr), (slope, intercept, r) = stats.probplot(std_resid.dropna(),
                                    dist=stats.t, sparams=(nu,))
axes[2,0].scatter(osm, osr, color='steelblue', s=8, alpha=0.6, label='Std Residuals')
axes[2,0].plot(osm, slope*np.array(osm)+intercept, color='red', lw=1.5,
        label=f'Student-t (v={nu:.1f})')
axes[2,0].set_title('QQ Plot – Std. Residuals vs Student-t')
axes[2,0].legend(); axes[2,0].grid(True, alpha=0.3)

axes[2,1].plot(returns.index[-30:], cond_vol.values[-30:],
        color='darkorange', lw=1.5, label='Historical Vol')
fc_idx2 = pd.date_range(returns.index[-1], periods=11, freq='D')[1:]
axes[2,1].plot(fc_idx2, fc_vol.values, color='red', lw=2, ls='--',
        marker='o', ms=5, label='10-Day Forecast')
axes[2,1].set_title('10-Day Volatility Forecast')
axes[2,1].legend(); axes[2,1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('Part2_Fig6_GARCH.png', dpi=150, bbox_inches='tight')
```

```
plt.show()
print("Figure 6 saved.")

# ── FIGURE 7: ARIMA vs GARCH Comparison ──────────────────────
fig7, axes = plt.subplots(2, 2, figsize=(16, 10))
fig7.suptitle('Figure 7: ARIMA vs GARCH — Comparative Summary\nBitcoin (BTC-USD)',
              fontsize=14, fontweight='bold')

x = np.arange(2)
b1 = axes[0,0].bar(x-0.2, [arima_model.aic, garch_res.aic], 0.35,
                   label='AIC', color='steelblue', alpha=0.85)
b2 = axes[0,0].bar(x+0.2, [arima_model.bic, garch_res.bic], 0.35,
                   label='BIC', color='purple', alpha=0.85)
axes[0,0].set_xticks(x)
axes[0,0].set_xticklabels(['ARIMA(1,0,1)', 'GARCH(1,1)-t'])
axes[0,0].set_title('AIC / BIC Comparison')
axes[0,0].legend(); axes[0,0].grid(True, alpha=0.3, axis='y')

lb_pvals  = [lb_arima['lb_pvalue'].values[0],
             lb_garch['lb_pvalue'].values[0],
             lb_sq['lb_pvalue'].values[0]]
col_bars  = ['green' if p > 0.05 else 'red' for p in lb_pvals]
bars = axes[0,1].bar(['ARIMA\nResid', 'GARCH\nStd', 'GARCH\nSq'],
                     lb_pvals, color=col_bars, alpha=0.85)
axes[0,1].axhline(0.05, color='red', lw=1.5, ls='--', label='5% threshold')
axes[0,1].set_title('Ljung-Box p-values (Q=10)')
axes[0,1].set_ylim(0, 1)
axes[0,1].legend(); axes[0,1].grid(True, alpha=0.3, axis='y')
for bar, p in zip(bars, lb_pvals):
    axes[0,1].text(bar.get_x()+bar.get_width()/2,
```

```
            bar.get_height()+0.02, f'{p:.3f}',
            ha='center', fontsize=9)

axes[1,0].plot(returns.iloc[-30:].index, returns.iloc[-30:].values*100,
        color='steelblue', lw=1.2, label='Actual')
axes[1,0].plot(fc_idx, fc_mean.values*100, color='red', lw=2, ls='--',
        marker='D', ms=4, label='ARIMA Forecast')
axes[1,0].fill_between(fc_idx, fc_ci.iloc[:,0]*100, fc_ci.iloc[:,1]*100,
            color='red', alpha=0.15, label='95% CI')
axes[1,0].set_title('ARIMA – 10-Day Return Forecast')
axes[1,0].legend(); axes[1,0].grid(True, alpha=0.3)

axes[1,1].plot(returns.index[-30:], cond_vol.values[-30:],
        color='darkorange', lw=1.5, label='Historical Vol')
axes[1,1].plot(fc_idx2, fc_vol.values, color='red', lw=2, ls='--',
        marker='D', ms=4, label='GARCH Forecast')
axes[1,1].set_title('GARCH – 10-Day Volatility Forecast')
axes[1,1].legend(); axes[1,1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('Part2_Fig7_Comparison.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure 7 saved.")

print("\nPART II COMPLETE")

 PART III: Algorithmic Trading Strategy Design

print("\n" + "=" * 60)
print("PART III: ALGORITHMIC TRADING STRATEGY DESIGN")
```

```
print("=" * 60)

# Data — Jan 2025 to Jan 2026
df = yf.download('BTC-USD', start='2025-01-01', end='2026-01-01',
                 auto_adjust=True, progress=False)
df = df[['Close']].copy()
df.columns = ['Close']
df['Return'] = df['Close'].pct_change()
df.dropna(inplace=True)

TRANSACTION_COST = 0.001
STOP_LOSS        = 0.02
TAKE_PROFIT      = 0.05
INITIAL_CAPITAL  = 10000

print(f"Data: {len(df)} trading days")
print(f"Period: {df.index[0].date()} to {df.index[-1].date()}")

# ── STRATEGY 1: MA Crossover ──────────────────────────────
df['MA20'] = df['Close'].rolling(20).mean()
df['MA50'] = df['Close'].rolling(50).mean()
df['Signal_MA'] = 0
df.loc[df['MA20'] > df['MA50'], 'Signal_MA'] = 1
df.loc[df['MA20'] < df['MA50'], 'Signal_MA'] = -1
df['Position_MA'] = df['Signal_MA'].diff()

capital_ma = INITIAL_CAPITAL
capital_ma_list = []
position = 0; entry_price = 0
```

```
for i in range(len(df)):
    price  = float(df['Close'].iloc[i])
    signal = int(df['Signal_MA'].iloc[i])
    if position == 1 and entry_price > 0:
        pnl = (price - entry_price) / entry_price
        if pnl <= -STOP_LOSS:
            capital_ma *= (1 + pnl - TRANSACTION_COST)
            position = 0; entry_price = 0
        elif pnl >= TAKE_PROFIT:
            capital_ma *= (1 + pnl - TRANSACTION_COST)
            position = 0; entry_price = 0
    if signal == 1 and position == 0:
        position = 1; entry_price = price
        capital_ma *= (1 - TRANSACTION_COST)
    elif signal != 1 and position == 1:
        pnl = (price - entry_price) / entry_price
        capital_ma *= (1 + pnl - TRANSACTION_COST)
        position = 0; entry_price = 0
    if position == 1 and entry_price > 0:
        capital_ma_list.append(capital_ma * (1 + (price-entry_price)/entry_price))
    else:
        capital_ma_list.append(capital_ma)

df['Capital_MA'] = capital_ma_list
df['Capital_BH'] = INITIAL_CAPITAL * (df['Close'] / float(df['Close'].iloc[0]))

ma_return    = (df['Capital_MA'].iloc[-1] - INITIAL_CAPITAL) / INITIAL_CAPITAL * 100
bh_return    = (df['Capital_BH'].iloc[-1] - INITIAL_CAPITAL) / INITIAL_CAPITAL * 100
ma_daily_ret = df['Capital_MA'].pct_change().dropna()
ma_sharpe    = (ma_daily_ret.mean() / ma_daily_ret.std()) * np.sqrt(365)
```

```
ma_max_dd    = ((df['Capital_MA'] - df['Capital_MA'].cummax()) /
df['Capital_MA'].cummax()).min() * 100

trades_ma    = int(df['Position_MA'].abs().sum())


print("\n" + "=" * 60)

print("STRATEGY 1 — TREND FOLLOWING (MA Crossover)")

print("=" * 60)

print(f"Total Return      : {ma_return:.2f}%")

print(f"Buy & Hold Return : {bh_return:.2f}%")

print(f"Sharpe Ratio      : {ma_sharpe:.4f}")

print(f"Max Drawdown      : {ma_max_dd:.2f}%")

print(f"Number of Trades  : {trades_ma}")


# ── STRATEGY 2: RSI ──────────────────────────────────────────

RSI_PERIOD     = 14

RSI_OVERSOLD   = 30

RSI_OVERBOUGHT = 70

TAKE_PROFIT_RSI = 0.04


delta = df['Close'].diff()

gain  = delta.where(delta > 0, 0).rolling(RSI_PERIOD).mean()

loss  = (-delta.where(delta < 0, 0)).rolling(RSI_PERIOD).mean()

rs    = gain / loss

df['RSI'] = 100 - (100 / (1 + rs))

df['Signal_RSI'] = 0

df.loc[df['RSI'] < RSI_OVERSOLD,   'Signal_RSI'] = 1

df.loc[df['RSI'] > RSI_OVERBOUGHT, 'Signal_RSI'] = -1


capital_rsi = INITIAL_CAPITAL

capital_rsi_list = []
```

```
position_rsi = 0; entry_price_rsi = 0


for i in range(len(df)):
    price  = float(df['Close'].iloc[i])
    signal = int(df['Signal_RSI'].iloc[i])
    if position_rsi == 1 and entry_price_rsi > 0:
        pnl = (price - entry_price_rsi) / entry_price_rsi
        if pnl <= -STOP_LOSS:
            capital_rsi *= (1 + pnl - TRANSACTION_COST)
            position_rsi = 0; entry_price_rsi = 0
        elif pnl >= TAKE_PROFIT_RSI:
            capital_rsi *= (1 + pnl - TRANSACTION_COST)
            position_rsi = 0; entry_price_rsi = 0
    if signal == 1 and position_rsi == 0:
        position_rsi = 1; entry_price_rsi = price
        capital_rsi *= (1 - TRANSACTION_COST)
    elif signal == -1 and position_rsi == 1:
        pnl = (price - entry_price_rsi) / entry_price_rsi
        capital_rsi *= (1 + pnl - TRANSACTION_COST)
        position_rsi = 0; entry_price_rsi = 0
    if position_rsi == 1 and entry_price_rsi > 0:
        capital_rsi_list.append(capital_rsi * (1 + (price-entry_price_rsi)/entry_price_rsi))
    else:
        capital_rsi_list.append(capital_rsi)


df['Capital_RSI'] = capital_rsi_list
rsi_return    = (df['Capital_RSI'].iloc[-1] - INITIAL_CAPITAL) / INITIAL_CAPITAL * 100
rsi_daily_ret = df['Capital_RSI'].pct_change().dropna()
rsi_sharpe    = (rsi_daily_ret.mean() / rsi_daily_ret.std()) * np.sqrt(365)
```

```
rsi_max_dd    = ((df['Capital_RSI'] - df['Capital_RSI'].cummax()) /
df['Capital_RSI'].cummax()).min() * 100

trades_rsi    = int((df['Signal_RSI'].diff().abs() > 0).sum())


print("\n" + "=" * 60)

print("STRATEGY 2 — MEAN REVERSION (RSI Strategy)")

print("=" * 60)

print(f"Total Return      : {rsi_return:.2f}%")

print(f"Buy & Hold Return : {bh_return:.2f}%")

print(f"Sharpe Ratio      : {rsi_sharpe:.4f}")

print(f"Max Drawdown      : {rsi_max_dd:.2f}%")

print(f"Number of Trades  : {trades_rsi}")


# ── FIGURE 8: MA Strategy ──────────────────────────────────

fig8, axes = plt.subplots(3, 1, figsize=(16, 14))

fig8.suptitle('Figure 8: Strategy 1 — Trend-Following (MA Crossover)\nBitcoin (BTC-USD) 2025',

          fontsize=14, fontweight='bold')


axes[0].plot(df.index, df['Close'], color='steelblue', lw=1.2, label='BTC Price')

axes[0].plot(df.index, df['MA20'], color='orange', lw=1.5, ls='--', label='MA20')

axes[0].plot(df.index, df['MA50'], color='purple', lw=1.5, ls='--', label='MA50')

buy_sig  = df[df['Position_MA'] == 2]

sell_sig = df[df['Position_MA'] == -2]

axes[0].scatter(buy_sig.index, buy_sig['Close'], marker='^', color='green', s=120, zorder=5,
label='Buy')

axes[0].scatter(sell_sig.index, sell_sig['Close'], marker='v', color='red', s=120, zorder=5,
label='Sell')

axes[0].set_title('BTC Price with MA20/MA50 & Trade Signals')

axes[0].set_ylabel('Price (USD)'); axes[0].legend(fontsize=9); axes[0].grid(True, alpha=0.3)
```

```
axes[1].fill_between(df.index, df['Signal_MA'], 0,
                     where=df['Signal_MA'] > 0, color='green', alpha=0.4, label='Long')
axes[1].fill_between(df.index, df['Signal_MA'], 0,
                     where=df['Signal_MA'] < 0, color='red', alpha=0.4, label='Out')
axes[1].plot(df.index, df['Signal_MA'], color='black', lw=0.8)
axes[1].set_title('Position Signal (1 = Long, -1 = Out)')
axes[1].set_ylabel('Signal'); axes[1].legend(); axes[1].grid(True, alpha=0.3)

axes[2].plot(df.index, df['Capital_MA'], color='green', lw=2, label='MA Strategy')
axes[2].plot(df.index, df['Capital_BH'], color='steelblue', lw=1.5, ls='--', label='Buy & Hold')
axes[2].axhline(INITIAL_CAPITAL, color='black', lw=0.8, ls=':')
axes[2].set_title('Equity Curve — MA Strategy vs Buy & Hold')
axes[2].set_ylabel('Portfolio Value ($)'); axes[2].legend(); axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('Part3_Fig8_MA_Strategy.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure 8 saved.")

# ── FIGURE 9: RSI Strategy ──────────────────────────────────────
fig9, axes = plt.subplots(3, 1, figsize=(16, 14))
fig9.suptitle('Figure 9: Strategy 2 — Mean-Reversion (RSI Strategy)\nBitcoin (BTC-USD) 2025',
              fontsize=14, fontweight='bold')

axes[0].plot(df.index, df['Close'], color='steelblue', lw=1.2, label='BTC Price')
rsi_buy  = df[df['Signal_RSI'] == 1]
rsi_sell = df[df['Signal_RSI'] == -1]
axes[0].scatter(rsi_buy.index, rsi_buy['Close'], marker='^', color='green', s=120, zorder=5,
label='Buy (RSI<30)')
```

```
axes[0].scatter(rsi_sell.index, rsi_sell['Close'], marker='v', color='red', s=120, zorder=5,
label='Sell (RSI>70)')

axes[0].set_title('BTC Price with RSI Buy/Sell Signals')

axes[0].set_ylabel('Price (USD)'); axes[0].legend(); axes[0].grid(True, alpha=0.3)


axes[1].plot(df.index, df['RSI'], color='darkorange', lw=1.5, label='RSI(14)')

axes[1].axhline(70, color='red', lw=1.5, ls='--', label='Overbought (70)')

axes[1].axhline(30, color='green', lw=1.5, ls='--', label='Oversold (30)')

axes[1].axhline(50, color='black', lw=0.8, ls=':', alpha=0.5)

axes[1].fill_between(df.index, df['RSI'], 70, where=df['RSI'] >= 70, color='red', alpha=0.3)

axes[1].fill_between(df.index, df['RSI'], 30, where=df['RSI'] <= 30, color='green', alpha=0.3)

axes[1].set_title('RSI(14) Indicator')

axes[1].set_ylabel('RSI'); axes[1].set_ylim(0, 100); axes[1].legend(); axes[1].grid(True,
alpha=0.3)


axes[2].plot(df.index, df['Capital_RSI'], color='purple', lw=2, label='RSI Strategy')

axes[2].plot(df.index, df['Capital_BH'], color='steelblue', lw=1.5, ls='--', label='Buy & Hold')

axes[2].axhline(INITIAL_CAPITAL, color='black', lw=0.8, ls=':')

axes[2].set_title('Equity Curve — RSI Strategy vs Buy & Hold')

axes[2].set_ylabel('Portfolio Value ($)'); axes[2].legend(); axes[2].grid(True, alpha=0.3)


plt.tight_layout()

plt.savefig('Part3_Fig9_RSI_Strategy.png', dpi=150, bbox_inches='tight')

plt.show()

print("Figure 9 saved.")


# ── FIGURE 10: Strategy Comparison ──────────────────────────

fig10, axes = plt.subplots(1, 3, figsize=(18, 6))

fig10.suptitle('Figure 10: Strategy Comparison — MA vs RSI vs Buy & Hold\nBitcoin (BTC-
USD) 2025',

        fontsize=13, fontweight='bold')
```

```
strategies = ['MA Crossover', 'RSI Strategy', 'Buy & Hold']
ret_vals   = [ma_return, rsi_return, bh_return]
sh_vals    = [ma_sharpe, rsi_sharpe, 0]
dd_vals    = [ma_max_dd, rsi_max_dd,
          ((df['Capital_BH'] - df['Capital_BH'].cummax()) /
df['Capital_BH'].cummax()).min()*100]
col3       = ['green', 'purple', 'steelblue']

for ax, vals, title, ylabel in zip(
   axes,
   [ret_vals, sh_vals, dd_vals],
   ['Total Return (%)', 'Sharpe Ratio', 'Max Drawdown (%)'],
   ['Return (%)', 'Sharpe', 'Drawdown (%)']
):
   bars = ax.bar(strategies, vals, color=col3, alpha=0.85, edgecolor='white')
   ax.axhline(0, color='black', lw=0.8, ls='--')
   ax.set_title(title, fontweight='bold')
   ax.set_ylabel(ylabel)
   ax.grid(True, alpha=0.3, axis='y')
   for bar, v in zip(bars, vals):
      ax.text(bar.get_x()+bar.get_width()/2,
            bar.get_height() + (0.3 if v >= 0 else -1.5),
            f'{v:.2f}', ha='center', fontsize=9, fontweight='bold')

plt.tight_layout()
plt.savefig('Part3_Fig10_StrategyComparison.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure 10 saved.")
```

```
print(f"\n{'='*60}")
print("PART III SUMMARY TABLE")
print(f"{'='*60}")
print(f"{'Metric':<25} {'MA Strategy':>12} {'RSI Strategy':>12} {'Buy & Hold':>12}")
print("-"*63)
print(f"{'Total Return (%)':<25} {ma_return:>12.2f} {rsi_return:>12.2f} {bh_return:>12.2f}")
print(f"{'Sharpe Ratio':<25} {ma_sharpe:>12.4f} {rsi_sharpe:>12.4f} {'N/A':>12}")
print(f"{'Max Drawdown (%)':<25} {ma_max_dd:>12.2f} {rsi_max_dd:>12.2f} {'N/A':>12}")
print(f"{'No. of Trades':<25} {trades_ma:>12} {trades_rsi:>12} {'1':>12}")

print("\nPART III COMPLETE")
```

PART IV: Backtesting, Performance Evaluation

```
print("\n" + "=" * 60)
print("PART IV: BACKTESTING AND PERFORMANCE EVALUATION")
print("=" * 60)

# Data — Jan 2024 to Jan 2026
df2 = yf.download('BTC-USD', start='2024-01-01', end='2026-01-01',
                  auto_adjust=True, progress=False)
df2 = df2[['Close']].copy()
df2.columns = ['Close']
df2['Return'] = df2['Close'].pct_change()
df2.dropna(inplace=True)

split_date = '2025-07-01'
df_in  = df2[df2.index < split_date].copy()
df_out = df2[df2.index >= split_date].copy()
```

```
print(f"In-Sample  : {df_in.index[0].date()} to {df_in.index[-1].date()} ({len(df_in)} days)")
print(f"Out-Sample : {df_out.index[0].date()} to {df_out.index[-1].date()} ({len(df_out)} days)")


# ── Backtest Functions ──────────────────────────────────────
def backtest_ma(data, short_w=20, long_w=50, sl=0.02, tp=0.05):
    d = data.copy()
    d['MA_s'] = d['Close'].rolling(short_w).mean()
    d['MA_l'] = d['Close'].rolling(long_w).mean()
    d['Sig']  = np.where(d['MA_s'] > d['MA_l'], 1, -1)
    capital = INITIAL_CAPITAL; cap_list = []
    pos = 0; ep = 0; wins = 0; losses = 0
    for i in range(len(d)):
        price = float(d['Close'].iloc[i])
        sig   = int(d['Sig'].iloc[i])
        if pos == 1 and ep > 0:
            pnl = (price - ep) / ep
            if pnl <= -sl:
                capital *= (1 + pnl - TRANSACTION_COST); losses += 1; pos = 0; ep = 0
            elif pnl >= tp:
                capital *= (1 + pnl - TRANSACTION_COST); wins += 1; pos = 0; ep = 0
        if sig == 1 and pos == 0:
            pos = 1; ep = price; capital *= (1 - TRANSACTION_COST)
        elif sig != 1 and pos == 1:
            pnl = (price - ep) / ep
            capital *= (1 + pnl - TRANSACTION_COST)
            if pnl > 0: wins += 1
            else: losses += 1
            pos = 0; ep = 0
        cap_list.append(capital * (1 + (price-ep)/ep) if pos == 1 and ep > 0 else capital)
```

```
    equity   = pd.Series(cap_list, index=d.index)
    ret      = (equity.iloc[-1] - INITIAL_CAPITAL) / INITIAL_CAPITAL * 100
    dr       = equity.pct_change().dropna()
    sharpe   = (dr.mean() / dr.std()) * np.sqrt(365) if dr.std() > 0 else 0
    max_dd   = ((equity - equity.cummax()) / equity.cummax()).min() * 100
    bh       = (float(d['Close'].iloc[-1]) - float(d['Close'].iloc[0])) / float(d['Close'].iloc[0]) * 100
    total_t  = wins + losses
    win_rate = wins / total_t * 100 if total_t > 0 else 0
    vol      = dr.std() * np.sqrt(365) * 100
    return {'equity': equity, 'total_return': ret, 'sharpe': sharpe,
            'max_dd': max_dd, 'bh_return': bh, 'win_rate': win_rate,
            'trades': total_t, 'volatility': vol,
            'calmar': abs(ret / max_dd) if max_dd != 0 else 0}

def backtest_rsi(data, period=14, oversold=30, overbought=70, sl=0.02, tp=0.04):
    d = data.copy()
    delta = d['Close'].diff()
    gain  = delta.where(delta > 0, 0).rolling(period).mean()
    loss  = (-delta.where(delta < 0, 0)).rolling(period).mean()
    d['RSI'] = 100 - (100 / (1 + gain / loss))
    d['Sig'] = np.where(d['RSI'] < oversold, 1, np.where(d['RSI'] > overbought, -1, 0))
    capital = INITIAL_CAPITAL; cap_list = []
    pos = 0; ep = 0; wins = 0; losses = 0
    for i in range(len(d)):
        price = float(d['Close'].iloc[i])
        sig   = int(d['Sig'].iloc[i])
        if pos == 1 and ep > 0:
            pnl = (price - ep) / ep
            if pnl <= -sl:
                capital *= (1 + pnl - TRANSACTION_COST); losses += 1; pos = 0; ep = 0
```

```
            elif pnl >= tp:
                capital *= (1 + pnl - TRANSACTION_COST); wins += 1; pos = 0; ep = 0
        if sig == 1 and pos == 0:
            pos = 1; ep = price; capital *= (1 - TRANSACTION_COST)
        elif sig == -1 and pos == 1:
            pnl = (price - ep) / ep
            capital *= (1 + pnl - TRANSACTION_COST)
            if pnl > 0: wins += 1
            else: losses += 1
            pos = 0; ep = 0
        cap_list.append(capital * (1 + (price-ep)/ep) if pos == 1 and ep > 0 else capital)
    equity   = pd.Series(cap_list, index=d.index)
    ret      = (equity.iloc[-1] - INITIAL_CAPITAL) / INITIAL_CAPITAL * 100
    dr       = equity.pct_change().dropna()
    sharpe   = (dr.mean() / dr.std()) * np.sqrt(365) if dr.std() > 0 else 0
    max_dd   = ((equity - equity.cummax()) / equity.cummax()).min() * 100
    bh       = (float(d['Close'].iloc[-1]) - float(d['Close'].iloc[0])) / float(d['Close'].iloc[0]) * 100
    total_t  = wins + losses
    win_rate = wins / total_t * 100 if total_t > 0 else 0
    vol      = dr.std() * np.sqrt(365) * 100
    return {'equity': equity, 'total_return': ret, 'sharpe': sharpe,
            'max_dd': max_dd, 'bh_return': bh, 'win_rate': win_rate,
            'trades': total_t, 'volatility': vol,
            'calmar': abs(ret / max_dd) if max_dd != 0 else 0}

# ── Run Backtests ──────────────────────────────────────────────
ma_in   = backtest_ma(df_in)
ma_out  = backtest_ma(df_out)
rsi_in  = backtest_rsi(df_in)
rsi_out = backtest_rsi(df_out)
```

```
# ── Walk-Forward Analysis ──────────────────────────────────────
wf_results = []
for start in range(0, len(df2) - 120 - 30, 30):
    test  = df2.iloc[start + 120 : start + 120 + 30]
    if len(test) < 5: break
    wf_results.append({
        'period_start': test.index[0],
        'ma_return':    backtest_ma(test)['total_return'],
        'rsi_return':   backtest_rsi(test)['total_return'],
        'bh_return':    backtest_ma(test)['bh_return']
    })
wf_df = pd.DataFrame(wf_results)

# ── Print Summary Tables ───────────────────────────────────────
print(f"\n{'='*65}")
print("IN-SAMPLE RESULTS (Jan 2024 – Jun 2025)")
print(f"{'='*65}")
print(f"{'Metric':<28} {'MA Strategy':>12} {'RSI Strategy':>12} {'Buy&Hold':>10}")
print("-"*65)
for metric, ma_v, rsi_v, bh_v in [
    ('Total Return (%)',    ma_in['total_return'],  rsi_in['total_return'],  ma_in['bh_return']),
    ('Sharpe Ratio',        ma_in['sharpe'],        rsi_in['sharpe'],        None),
    ('Max Drawdown (%)',    ma_in['max_dd'],        rsi_in['max_dd'],        None),
    ('Win Rate (%)',        ma_in['win_rate'],      rsi_in['win_rate'],      None),
    ('Annualised Vol (%)',  ma_in['volatility'],    rsi_in['volatility'],    None),
    ('Calmar Ratio',        ma_in['calmar'],        rsi_in['calmar'],        None),
    ('No. of Trades',       ma_in['trades'],        rsi_in['trades'],        None),
]:
    bh_str = f'{bh_v:>10.2f}' if bh_v is not None else f"{'N/A':>10}"
```

```
    print(f"{metric:<28} {ma_v:>12.2f} {rsi_v:>12.2f} {bh_str}")


print(f"\n{'='*65}")
print("OUT-OF-SAMPLE RESULTS (Jul 2025 – Jan 2026)")
print(f"{'='*65}")
print(f"{'Metric':<28} {'MA Strategy':>12} {'RSI Strategy':>12} {'Buy&Hold':>10}")
print("-"*65)
for metric, ma_v, rsi_v, bh_v in [
    ('Total Return (%)',    ma_out['total_return'],  rsi_out['total_return'],
ma_out['bh_return']),
    ('Sharpe Ratio',        ma_out['sharpe'],        rsi_out['sharpe'],       None),
    ('Max Drawdown (%)',    ma_out['max_dd'],        rsi_out['max_dd'],       None),
    ('Win Rate (%)',        ma_out['win_rate'],      rsi_out['win_rate'],     None),
    ('Annualised Vol (%)',  ma_out['volatility'],    rsi_out['volatility'],   None),
    ('Calmar Ratio',        ma_out['calmar'],        rsi_out['calmar'],       None),
    ('No. of Trades',       ma_out['trades'],        rsi_out['trades'],       None),
]:
    bh_str = f'{bh_v:>10.2f}' if bh_v is not None else f"{'N/A':>10}"
    print(f"{metric:<28} {ma_v:>12.2f} {rsi_v:>12.2f} {bh_str}")


print(f"\nWalk-Forward Windows    : {len(wf_df)}")
print(f"MA  Avg Window Return   : {wf_df['ma_return'].mean():.2f}%")
print(f"RSI Avg Window Return   : {wf_df['rsi_return'].mean():.2f}%")
print(f"B&H Avg Window Return   : {wf_df['bh_return'].mean():.2f}%")


# ── FIGURE 11: In-Sample vs Out-of-Sample Equity Curves ──────
fig11, axes = plt.subplots(2, 2, figsize=(18, 12))
fig11.suptitle('Figure 11: Backtesting — In-Sample vs Out-of-Sample\nBitcoin (BTC-USD)',
               fontsize=14, fontweight='bold')
```

```
bh_in  = INITIAL_CAPITAL * (df_in['Close'] / float(df_in['Close'].iloc[0]))
bh_out = INITIAL_CAPITAL * (df_out['Close'] / float(df_out['Close'].iloc[0]))

for ax, equity, bh, title, color in [
   (axes[0,0], ma_in['equity'],  bh_in,  'MA — In-Sample (Jan 2024–Jun 2025)', 'green'),
   (axes[0,1], ma_out['equity'], bh_out, 'MA — Out-of-Sample (Jul–Jan 2026)',  'green'),
   (axes[1,0], rsi_in['equity'], bh_in,  'RSI — In-Sample (Jan 2024–Jun 2025)','purple'),
   (axes[1,1], rsi_out['equity'],bh_out, 'RSI — Out-of-Sample (Jul–Jan 2026)', 'purple'),
]:
   ax.plot(equity.index, equity.values, color=color, lw=2, label='Strategy')
   ax.plot(bh.index, bh.values, color='steelblue', lw=1.5, ls='--', label='Buy & Hold')
   ax.axhline(INITIAL_CAPITAL, color='black', lw=0.6, ls=':')
   ax.set_title(title, fontweight='bold')
   ax.set_ylabel('Portfolio Value ($)')
   ax.legend(fontsize=9); ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('Part4_Fig11_InOut_Sample.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure 11 saved.")

# ── FIGURE 12: Walk-Forward Returns ──────────────────────────
fig12, axes = plt.subplots(2, 1, figsize=(16, 10))
fig12.suptitle('Figure 12: Walk-Forward Analysis — Rolling Windows\nBitcoin (BTC-USD)',
               fontsize=14, fontweight='bold')

for ax, col, label, title in [
   (axes[0], 'steelblue', 'MA Return',  'MA Crossover — Return per Rolling Window'),
   (axes[1], 'purple',    'RSI Return', 'RSI Strategy — Return per Rolling Window'),
]:
```

```
    key = 'ma_return' if 'MA' in title else 'rsi_return'
    ax.bar(range(len(wf_df)), wf_df[key],
           color=[col if r > 0 else 'red' for r in wf_df[key]],
           alpha=0.85, edgecolor='white', lw=0.3)
    ax.plot(range(len(wf_df)), wf_df['bh_return'],
            color='black', lw=1.5, ls='--', marker='o', ms=4, label='Buy & Hold')
    ax.axhline(0, color='black', lw=0.8, ls=':')
    ax.set_title(title, fontweight='bold')
    ax.set_ylabel('Return (%)'); ax.legend(); ax.grid(True, alpha=0.3, axis='y')
    ax.set_xticks(range(len(wf_df)))
    ax.set_xticklabels([str(d.date()) for d in wf_df['period_start']],
                       rotation=45, fontsize=7)

plt.tight_layout()
plt.savefig('Part4_Fig12_WalkForward.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figure 12 saved.")

# ── FIGURE 13: Performance Metrics Comparison ──────────────────
fig13, axes = plt.subplots(2, 3, figsize=(18, 10))
fig13.suptitle('Figure 13: Performance Metrics — In-Sample vs Out-of-Sample\nBitcoin (BTC-USD)',
               fontsize=14, fontweight='bold')

labels  = ['MA\nIn', 'MA\nOut', 'RSI\nIn', 'RSI\nOut']
colors4 = ['green', '#90EE90', 'purple', '#DDA0DD']
metrics_data = [
    ([ma_in['total_return'], ma_out['total_return'], rsi_in['total_return'],
rsi_out['total_return']], 'Total Return (%)',         'Return (%)'),
    ([ma_in['sharpe'],      ma_out['sharpe'],      rsi_in['sharpe'],      rsi_out['sharpe']],
'Sharpe Ratio',            'Sharpe'),
```

```
   ([ma_in['max_dd'],      ma_out['max_dd'],      rsi_in['max_dd'],      rsi_out['max_dd']],
'Max Drawdown (%)',         'Drawdown (%)'),

   ([ma_in['win_rate'],    ma_out['win_rate'],    rsi_in['win_rate'],    rsi_out['win_rate']],
'Win Rate (%)',             'Win Rate (%)'),

   ([ma_in['volatility'],  ma_out['volatility'],  rsi_in['volatility'],  rsi_out['volatility']],
'Annualised Volatility (%)', 'Vol (%)'),

   ([ma_in['calmar'],      ma_out['calmar'],      rsi_in['calmar'],      rsi_out['calmar']],
'Calmar Ratio',             'Calmar'),

]


for ax, (vals, title, ylabel) in zip(axes.flat, metrics_data):

   bars = ax.bar(labels, vals, color=colors4, alpha=0.85, edgecolor='white', lw=0.5)

   ax.axhline(0, color='black', lw=0.6, ls='--')

   ax.set_title(title, fontweight='bold')

   ax.set_ylabel(ylabel); ax.grid(True, alpha=0.3, axis='y')

   for bar, v in zip(bars, vals):

      ax.text(bar.get_x()+bar.get_width()/2,

            bar.get_height() + (0.3 if v >= 0 else -1.5),

            f'{v:.2f}', ha='center', fontsize=8, fontweight='bold')


plt.tight_layout()

plt.savefig('Part4_Fig13_PerformanceMetrics.png', dpi=150, bbox_inches='tight')

plt.show()

print("Figure 13 saved.")

print("\n" + "=" * 60)

print("ALL PARTS COMPLETE — ALL FIGURES SAVED")

print("=" * 60)

print("Part I  : Part1_Fig1–Fig4")

print("Part II : Part2_Fig5–Fig7")

print("Part III: Part3_Fig8–Fig10")

print("Part IV : Part4_Fig11–Fig13")
```